

Projet LSF (FR) → SigML → SiGML-Player : description des fichiers et commandes

Vue d'ensemble

Ce projet permet :

- de convertir une phrase en français en une **séquence de signes** (fichiers `.sigml`);
- d'envoyer ces signes au **SiGML-Player** (lecture de l'avatar) via une connexion TCP (par défaut `127.0.0.1:8052`);
- d'utiliser la **reconnaissance vocale** (Vosk) pour passer de la voix au texte, puis du texte aux signes.

Organisation du dossier

À la racine du dossier projet :

- `LexData.xls` : lexique (souvent un TSV UTF-16 malgré l'extension);
- `sigml_fr/` : collection locale de signes `.sigml` (noms en français) + alphabet `a.sigml...z.sigml`;
- `SiGML-Player.exe` : lecteur d'avatar;
- scripts Python (voir ci-dessous).

Description des fichiers

`phrase_to_sigml_sequence.py`

Rôle : coeur logique « texte → plan de signes ».

- Lecture du lexique `LexData.xls`.
- Tokenisation et normalisation du texte.
- Segmentation par **plus longue expression** possible (matching multi-mots).
- Production d'une liste de **Steps** : **SIGN** (jouer un `.sigml`), **SPELL** (épeler), **SKIP** (ignorer).
- Peut être exécuté en CLI pour produire un plan (`-out`) et l'afficher (`-show_plan`).

`lsf_context.py`

Rôle : construire le **contexte** (lexique + index local) et exposer les pipelines haut niveau.

- Chargement du lexique et construction d'une structure de recherche (trie).
- Indexation du dossier `sigml_fr/` (fichiers disponibles localement).
- Fonctions principales : `make_context`, `build_steps_basic` (texte), `build_steps_voice` (voix).

`lsf_play.py`

Rôle : exécuter une liste de **Steps** et jouer les signes dans le bon ordre.

- Charge les `.sigml` des **SIGN**.
- Pour **SPELL**, charge `a.sigml...z.sigml` (et éventuellement les chiffres).
- Envoie chaque signe au player avec des délais (`delay_sign` / `delay_letter`).

`sigml_player.py`

Rôle : communication réseau avec le SiGML-Player.

- `send` : envoi TCP d'un SigML (bytes) au player.
- `can_connect` : test de disponibilité sur `host:port`.
- `start_player` : lancement de `SiGML-Player.exe` si nécessaire.

`speech_vosk.py`

Rôle : reconnaître la parole (micro) et produire des événements de transcription.

- Charge un modèle Vosk.
- Lit le micro (via `sounddevice`).
- Expose un générateur de résultats `partial` et `final`.

`cli_phrase_play.py`

Rôle : point d'entrée « texte → signes → lecture ».

- Entrée : `-text`.
- Contexte : `make_context`.
- Steps : `build_steps_basic`.
- Lecture : `play_steps`.

`cli_voice_to_phrase.py`

Rôle : point d'entrée « voix → texte ».

- Lance l'écoute micro.
- Affiche `FINAL` (et `PARTIAL` si demandé).

`cli_voice_play.py`

Rôle : point d'entrée « voix → texte → signes → lecture ».

- Écoute le micro via `speech_vosk.py`.
- Pour chaque transcription `FINAL`, produit les Steps via `build_steps_voice`.
- Joue la séquence dans le SiGML-Player via `lsf_play.py`.

Commandes à exécuter

Installation (une fois)

```
python -m pip install --upgrade pip
pip install pandas
pip install vosk sounddevice
pip install spacy
python -m spacy download fr_core_news_sm
```

1) Générer un plan (texte -> Steps) sans lecture

```
python .\phrase_to_sigml_sequence.py --text "je mange" --out sequence.txt --show_plan
```

2) Texte -> lecture dans SiGML-Player

```
python .\cli_phrase_play.py --text "je mange" --show_plan --start_player '
--player_exe ".\SiGML-Player.exe"
```

3) Voix -> texte (transcription seule)

```
python .\cli_voice_to_phrase.py --show_partial
```

4) Voix -> LSF complet (voix -> texte -> signes -> lecture)

```
python .\cli_voice_play.py --start_player --player_exe ".\SiGML-Player.exe" '  
--show_partial --show_plan
```